

TP3 -- Structures de controle, fonctions et pointeurs

Matiere : Decouverte des langages compiles

Niveau : Bachelor 1

Duree : 4h (cours + atelier)

Jour : 3/5 — lundi 1er juin 2026

Objectifs pedagogiques couverts :

- Utiliser if/else et switch pour faire des choix
- Ecrire des boucles for, while et do...while sans boucle infinie
- Definir et appeler des fonctions avec parametres et retour
- Manipuler des tableaux et des chaines de caracteres
- Comprendre la difference passage par valeur / par adresse
- Lire et ecrire les operateurs `*`, `&` et utiliser des pointeurs simples

Contexte

Vous savez compiler un programme et declarer des variables. Maintenant on va apprendre a faire faire des choses interessantes a votre programme : prendre des decisions, repeter des actions, organiser le code en blocs reutilisables, et manipuler la memoire avec les pointeurs.

Partie 1 -- Faire des choix : conditions (45 min)

1.1 if / else if / else

```
#include <stdio.h>

int main(void) {
    int note;
    printf("Note (0-20) : ");
    scanf("%d", &note);

    if (note < 0 || note > 20) {
        printf("Note invalide\n");
    } else if (note >= 16) {
        printf("Tres bien\n");
    } else if (note >= 12) {
        printf("Bien\n");
    } else if (note >= 10) {
        printf("Passable\n");
    } else {
        printf("Insuffisant\n");
    }
    return 0;
}
```

Points importants :

- `==` est la comparaison, `=` est l'affectation. Les confondre est un bug classique.
- Les accolades `{ }` delimitent les blocs. Mettez-les toujours, meme pour une seule ligne.

1.2 switch / case

```
#include <stdio.h>

int main(void) {
    int jour;
    printf("Numero du jour (1-7) : ");
    scanf("%d", &jour);

    switch (jour) {
        case 1: printf("Lundi\n");    break;
        case 2: printf("Mardi\n");   break;
        case 3: printf("Mercredi\n"); break;
        case 4: printf("Jeudi\n");    break;
        case 5: printf("Vendredi\n"); break;
        case 6:
        case 7: printf("Week-end\n"); break;
        default: printf("Jour invalide\n");
    }
    return 0;
}
```

Points importants :

- Le `break` est essentiel : sans lui, l'exécution "tombe" dans le case suivant (fall-through)
- `switch` ne fonctionne qu'avec des entiers (int, char, enum), pas avec des chaînes

1.3 Exercice : convertisseur de notes

Creez `notes.c` qui :

1. Demande une note sur 20
2. Affiche la mention correspondante (Tres bien, Bien, Assez bien, Passable, Insuffisant)
3. Affiche "Admis" si ≥ 10 , "Refuse" sinon
4. Gere les notes invalides (< 0 ou > 20)

Partie 2 -- Repeter : les boucles (45 min)

2.1 La boucle for

```
#include <stdio.h>

int main(void) {
    // Afficher les nombres de 1 a 10
    for (int i = 1; i <= 10; i++) {
        printf("%d ", i);
    }
    printf("\n");

    // Calculer la somme de 1 a 100
    int somme = 0;
    for (int i = 1; i <= 100; i++) {
        somme += i;
    }
    printf("Somme 1+2+...+100 = %d\n", somme);

    return 0;
}
```

2.2 Les boucles while et do...while

```
#include <stdio.h>

int main(void) {
    // while : teste AVANT chaque execution
    int compteur = 1;
    while (compteur <= 5) {
        printf("Tour %d\n", compteur);
        compteur++;    // sans cela : boucle infinie !
    }

    // do...while : execute AU MOINS une fois
    int reponse;
    do {
        printf("Entrez 0 pour quitter : ");
        scanf("%d", &reponse);
    } while (reponse != 0);

    printf("Au revoir\n");
    return 0;
}
```

Piege classique : oublier d'incrémenter le compteur = boucle infinie.
Utilisez Ctrl+C pour arreter.

2.3 Exercice : table de multiplication

Creez `table.c` qui :

1. Demande un nombre `n` a l'utilisateur
2. Affiche la table de multiplication de `n` (de 1 a 10)
3. Redemande un nombre tant que l'utilisateur ne saisit pas 0

Exemple de sortie :

```
Nombre (0 pour quitter) : 7
7 x 1  = 7
7 x 2  = 14
...
7 x 10 = 70
Nombre (0 pour quitter) : 0
Au revoir !
```

Partie 3 -- Les fonctions (45 min)

3.1 Anatomie d'une fonction

```
#include <stdio.h>

// Prototype : annonce la fonction au compilateur
int carre(int x);

// Definition : implementation reelle
int carre(int x) {
    return x * x;
}

int main(void) {
    int a = carre(5);           // a = 25
    int b = carre(7);           // b = 49
    printf("5*5=%d, 7*7=%d\n", a, b);
    printf("12*12=%d\n", carre(12));
    return 0;
}
```

Points importants :

- `type_retour nom(parametres) { ... return valeur; }`
- Le prototype (declaration sans corps) permet d'utiliser la fonction avant sa definition
- `void` en type de retour = la fonction ne retourne rien (comme une "procedure")

3.2 Passage par valeur -- le piège classique

```
#include <stdio.h>

void doubler(int x) {
    x = x * 2;
    // x est une COPIE locale, modifier x ne change rien dehors
}

int main(void) {
    int valeur = 10;
    doubler(valeur);
    printf("valeur = %d\n", valeur); // affiche 10 (et non 20)
    return 0;
}
```

3.3 Exercice : bibliotheque de fonctions

Creez `fonctions.c` avec les fonctions suivantes :

Fonction	Signature	Description
Minimum	<code>int minimum(int a, int b)</code>	Retourne le plus petit des deux
Maximum	<code>int maximum(int a, int b)</code>	Retourne le plus grand des deux
Valeur absolue	<code>int valeur_absolue(int x)</code>	Retourne la valeur absolue
Est pair	<code>int est_pair(int x)</code>	Retourne 1 si pair, 0 sinon
Factorielle	<code>long factorielle(int n)</code>	Calcule n! (iteratif avec une boucle)

Testez chaque fonction dans `main` avec `printf` .

Partie 4 -- Tableaux et chaines (30 min)

4.1 Les tableaux

```
#include <stdio.h>

int main(void) {
    int notes[5] = {14, 17, 9, 12, 18};

    // Acces par indice (commence a 0 !)
    printf("Premiere note : %d\n", notes[0]); // 14
    printf("Derniere note : %d\n", notes[4]); // 18

    // Parcours classique avec for
    int total = 0;
    for (int i = 0; i < 5; i++) {
        total += notes[i];
    }
    printf("Moyenne : %.2f\n", total / 5.0);

    return 0;
}
```

Points importants :

- Les indices commencent a **0**. Un tableau de taille **n** a des indices de **0** a **n-1**.
- Acceder a **notes[5]** sur un tableau de taille 5 = **comportement indefini** (bug grave).

4.2 Les chaines de caracteres

```
#include <stdio.h>
#include <string.h>    // pour strlen, strcpy, strcmp

int main(void) {
    char prenom[20] = "Ayoub";    // taille 20, contient 'A','y'

    printf("Bonjour %s\n", prenom);
    printf("Longueur : %zu\n", strlen(prenom));    // 5

    char autre[20];
    strcpy(autre, "Adam");        // copier
    if (strcmp(prenom, autre) == 0) {    // comparer
        printf("Memes prenoms\n");
    } else {
        printf("Prenoms differents\n");
    }

    return 0;
}
```

4.3 Exercice : statistiques de notes

Creez `statistiques.c` qui :

1. Declare un tableau de 8 notes (entiers) initialisees en dur
2. Calcule et affiche : la somme, la moyenne, la note minimum et la note maximum
3. Utilisez des fonctions separees pour chaque calcul

Partie 5 -- Introduction aux pointeurs (45 min)

5.1 Qu'est-ce qu'un pointeur ?

Un pointeur est une variable qui **stocke une adresse memoire**.

```

#include <stdio.h>

int main(void) {
    int x = 42;
    printf("Valeur de x : %d\n", x);
    printf("Adresse de x : %p\n", (void*)&x);

    // Declaration d'un pointeur
    int *p = &x;          // p contient l'adresse de x

    // * = dereferencement (accéder à la valeur pointée)
    printf("*p vaut : %d\n", *p);    // 42

    // Modifier x via le pointeur
    *p = 100;
    printf("Maintenant x = %d\n", x); // 100 (modifié via *p)
    return 0;
}

```

Resume des operateurs :

- `&` devant une variable = donne son **adresse memoire**
- `*` a la declaration = declare un **pointeur** (`int *p`)
- `*` a l'utilisation = donne la **valeur pointee** (dereferencement)

5.2 Passage par adresse -- la solution

```
#include <stdio.h>

// Recoit l'ADRESSE d'un int, peut donc le modifier
void doubler(int *x) {
    *x = *x * 2;
}

// Echanger deux valeurs : impossible sans pointeurs
void echanger(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main(void) {
    int valeur = 10;
    doubler(&valeur);           // on passe l'adresse
    printf("valeur = %d\n", valeur); // 20

    int x = 1, y = 2;
    echanger(&x, &y);
    printf("x=%d, y=%d\n", x, y); // x=2, y=1
    return 0;
}
```

C'est exactement ce que fait scanf : on écrit `scanf("%d", &age)` pour que scanf puisse écrire dans la variable `age` .

5.3 Pointeurs et tableaux

Le nom d'un tableau est un **pointeur vers sa première case** :

```
int tab[5] = {10, 20, 30, 40, 50};
int *p = tab;    // p pointe vers tab[0]

// Ces notations sont equivalentes :
tab[2]    // 30
*(p + 2)  // 30
p[2]      // 30
```

5.4 Exercice : fonctions avec pointeurs

Creez `pointeurs.c` avec :

1. Une fonction `void min_max(int tab[], int n, int *pmin, int *pmax)` qui trouve le minimum et le maximum d'un tableau et les retourne via des pointeurs
2. Une fonction `double moyenne(int tab[], int n)` qui calcule la moyenne
3. Testez avec un tableau de 6 notes dans `main`

Pourquoi des pointeurs pour min_max ? En C, une fonction ne peut retourner qu'UNE seule valeur. Les pointeurs permettent d'en "retourner" plusieurs.

Partie 6 -- Mini-projet : devine le nombre (30 min)

Creez `devine.c` : le jeu classique ou l'ordinateur tire un nombre au hasard et l'utilisateur doit le trouver.

Regles :

1. Inclure `stdlib.h` et `time.h`
2. Initialiser le generateur aleatoire : `srand(time(NULL));`
3. Tirer un nombre entre 1 et 100 : `rand() % 100 + 1`

4. Boucler tant que l'utilisateur n'a pas trouve : lire son essai, dire "trop petit" ou "trop grand"
5. Compter le nombre de tentatives et l'afficher a la fin
6. Encapsuler la logique de jeu dans une fonction `int jouer(int secret)` qui retourne le nombre de tentatives

Exemple de partie :

```
=== Devine le nombre (1-100) ===  
Ton essai : 50  
Trop grand !  
Ton essai : 25  
Trop petit !  
Ton essai : 37  
Trop petit !  
Ton essai : 42  
Bravo ! Trouve en 4 tentatives.
```

Bonus : Proposez plusieurs niveaux de difficulté en variant la borne max (50, 100, 1000).

Pour aller plus loin

- Reécrivez `factorielle` de manière **recursive** (la fonction s'appelle elle-même). Quelle version est plus lisible ? Plus efficace ?
- Ajoutez une fonction `fibonacci(int n)` à votre bibliothèque
- Explorez `strcat` (concatenation) et `sprintf` (printf dans une chaîne)
- Lisez le chapitre 5 de *The C Programming Language* (Kernighan & Ritchie)

